

Pack 13 — Billing Hardening: Stripe-Ready Entitlements, Usage Limits & Audit-Safe Plan Enforcement (real code)

RemoteDesk · Pack 13 — Billing Hardening: Stripe-Ready Entitlements, Usage Limits & Audit-Safe Plan Enforcement (real code)

Codex / Claude Code handoff. Copy each file to the path in its heading. Build order: shared → api → web → desktop → tests. Every file is tagged **SAFE_DIRECT_COPY** (new file, drop in) or **REVIEW_REQUIRED** (patch to an existing file — merge by hand).

Scope: a Stripe-**ready** entitlements + usage-limit foundation with audit-safe plan enforcement. **No real Stripe secret keys. No fake payment completion. Stripe webhook signature verification is an isolated stub that runs without keys in dev, and failed verification is audit-logged without storing secrets. Plan enforcement fails CLOSED. Native input execution stays disabled.**

Zip-ready file tree

```
remotedesk/
  packages/shared/src/
    entitlements/
      types.ts                # SAFE_DIRECT_COPY
      planLimits.ts           # SAFE_DIRECT_COPY
      featureGate.ts          # SAFE_DIRECT_COPY
      usageWindow.ts          # SAFE_DIRECT_COPY
      schema.ts               # SAFE_DIRECT_COPY
      index.ts                # SAFE_DIRECT_COPY
    index.ts                  # REVIEW_REQUIRED (patch)
    errors/errorCodes.ts      # REVIEW_REQUIRED (patch)
  apps/api/
    prisma/schema.prisma      # REVIEW_REQUIRED (patch)
    prisma/migrations/0013_billing_entitlements/migration.sql  # SAFE_DIRECT_COPY
    src/lib/entitlements.ts     # SAFE_DIRECT_COPY
    src/lib/usageCounters.ts   # SAFE_DIRECT_COPY
    src/lib/billingAudit.ts    # SAFE_DIRECT_COPY
    src/lib/stripeSignature.ts # SAFE_DIRECT_COPY (isolated verify stub)
    src/repositories/billingRepository.ts # SAFE_DIRECT_COPY
    src/services/billing/entitlements.service.ts # SAFE_DIRECT_COPY
    src/services/billing/stripeWebhook.service.ts # SAFE_DIRECT_COPY
    src/routes/billing.routes.ts # SAFE_DIRECT_COPY
    src/server.ts              # REVIEW_REQUIRED (patch)
  apps/web/app/dashboard/billing/
    entitlements/page.tsx      # SAFE_DIRECT_COPY
    usage/page.tsx             # SAFE_DIRECT_COPY
    billingApi.ts              # SAFE_DIRECT_COPY
    billing.module.css         # SAFE_DIRECT_COPY
  apps/web/app/dashboard/page.tsx # REVIEW_REQUIRED (patch — add links)
  apps/desktop/src/renderer/src/
```

```

services/entitlementsClient.ts      # SAFE_DIRECT_COPY
services/featureGateClient.ts      # SAFE_DIRECT_COPY
session/sessionStart.ts            # REVIEW_REQUIRED (patch)
tests/entitlements/
  featureGate.test.ts              # SAFE_DIRECT_COPY
  planLimits.test.ts               # SAFE_DIRECT_COPY
  usageWindow.test.ts              # SAFE_DIRECT_COPY
tests/billing/
  webhook-signature-stub.test.ts   # SAFE_DIRECT_COPY
  usage-counter-contract.test.ts    # SAFE_DIRECT_COPY
docs/
  billing-entitlements-hardening.md # SAFE_DIRECT_COPY
generated-remotedesk-pack13-merge-summary.md # doNotMerge (handoff meta)
generated-remotedesk-pack13-code-review.md    # doNotMerge (handoff meta)
generated-remotedesk-pack13-manifest.json      # doNotMerge (handoff meta)

```

Part A — Shared package

packages/shared/src/entitlements/types.ts — SAFE_DIRECT_COPY

```

/** Billing plan tiers. Mirrors Stripe products but defined in code (no keys). */
export type PlanTier = 'free' | 'pro' | 'business' | 'enterprise';

/** Gateable product features. */
export type FeatureKey =
  | 'fileTransfer'
  | 'clipboardSync'
  | 'unattendedAccess'
  | 'sessionRecording'
  | 'multiMonitor'
  | 'auditExport';

/** Metered usage dimensions (counted per window). */
export type UsageMetric =
  | 'sessionsStarted'
  | 'concurrentSessions'
  | 'recordedMinutes'
  | 'fileTransferBytes';

/** A plan's full limit + feature definition. */
export interface PlanDefinition {
  tier: PlanTier;
  name: string;
  /** Features enabled at this tier. */
  features: FeatureKey[];
  /** Numeric caps per usage metric. null = unlimited. */
  limits: Record<UsageMetric, number | null>;
}

/** A resolved entitlement set for a subscriber (plan + any overrides). */
export interface EntitlementSet {
  tier: PlanTier;
  features: FeatureKey[];
  limits: Record<UsageMetric, number | null>;
}

```

```

    /** ISO; when the entitlement was resolved. */
    resolvedAt: string;
    /** True if this is the safe fallback (API/plan unknown). */
    failClosed: boolean;
}

/** A single usage counter reading. */
export interface UsageCounterDto {
    metric: UsageMetric;
    used: number;
    limit: number | null;
    /** ISO window bounds. */
    windowStart: string;
    windowEnd: string;
    /** True when used >= limit (and limit is not null). */
    exceeded: boolean;
}

/** Result of a feature gate check. */
export interface GateResult {
    feature: FeatureKey;
    allowed: boolean;
    reason: 'allowed' | 'not_in_plan' | 'fail_closed';
}

```

packages/shared/src/entitlements/planLimits.ts — SAFE_DIRECT_COPY

```

import type { PlanDefinition, PlanTier } from './types';

/**
 * Canonical plan catalog. Single source of truth for features + limits. The API
 * resolves entitlements against this; the web UI renders from it; tests assert
 * against it. Stripe product/price IDs are intentionally NOT here (no keys).
 */
export const PLAN_DEFINITIONS: Readonly<Record<PlanTier, PlanDefinition>> = Object.freeze({
    free: {
        tier: 'free',
        name: 'Free',
        features: ['clipboardSync'],
        limits: {
            sessionsStarted: 50,
            concurrentSessions: 1,
            recordedMinutes: 0,
            fileTransferBytes: 0,
        },
    },
    pro: {
        tier: 'pro',
        name: 'Pro',
        features: ['clipboardSync', 'fileTransfer', 'multiMonitor'],
        limits: {
            sessionsStarted: 1000,
            concurrentSessions: 3,
            recordedMinutes: 0,
            fileTransferBytes: 5_000_000_000,
        },
    },
});

```

```

business: {
  tier: 'business',
  name: 'Business',
  features: ['clipboardSync', 'fileTransfer', 'multiMonitor', 'unattendedAccess', 'sessionRecording', 'auditExport'],
  limits: {
    sessionsStarted: 10_000,
    concurrentSessions: 10,
    recordedMinutes: 5000,
    fileTransferBytes: 50_000_000_000,
  },
},
enterprise: {
  tier: 'enterprise',
  name: 'Enterprise',
  features: ['clipboardSync', 'fileTransfer', 'multiMonitor', 'unattendedAccess', 'sessionRecording', 'auditExport'],
  limits: {
    sessionsStarted: null,
    concurrentSessions: null,
    recordedMinutes: null,
    fileTransferBytes: null,
  },
},
});

/** The fail-closed plan: the most restrictive possible. Used when plan unknown. */
export const FAIL_CLOSED_TIER: PlanTier = 'free';

export function getPlan(tier: PlanTier): PlanDefinition {
  return PLAN_DEFINITIONS[tier] ?? PLAN_DEFINITIONS[FAIL_CLOSED_TIER];
}

```

packages/shared/src/entitlements/featureGate.ts — SAFE_DIRECT_COPY

```

import type { EntitlementSet, FeatureKey, GateResult, PlanTier } from './types';
import { getPlan } from './planLimits';

/**
 * Pure feature gating shared by server enforcement + client UI gating so they
 * never drift. Fails CLOSED: an unknown/fail-closed entitlement denies the
 * feature unless the safe fallback plan explicitly includes it.
 */
export function resolveEntitlements(
  tier: PlanTier,
  failClosed = false,
): EntitlementSet {
  const plan = getPlan(tier);
  return {
    tier: plan.tier,
    features: [...plan.features],
    limits: { ...plan.limits },
    resolvedAt: new Date().toISOString(),
    failClosed,
  };
}

export function checkFeature(
  entitlements: EntitlementSet,

```

```

    feature: FeatureKey,
  ): GateResult {
    if (entitlements.failClosed) {
      // Even fail-closed, honor features in the safe fallback plan only.
      const allowed = entitlements.features.includes(feature);
      return { feature, allowed, reason: allowed ? 'allowed' : 'fail_closed' };
    }
    const allowed = entitlements.features.includes(feature);
    return { feature, allowed, reason: allowed ? 'allowed' : 'not_in_plan' };
  }

  /** Convenience: which of a list of features are blocked. */
  export function blockedFeatures(
    entitlements: EntitlementSet,
    features: FeatureKey[],
  ): FeatureKey[] {
    return features.filter((f) => !checkFeature(entitlements, f).allowed);
  }

```

packages/shared/src/entitlements/usageWindow.ts — SAFE_DIRECT_COPY

```

import type { UsageCounterDto, UsageMetric } from './types';

/**
 * Pure usage-window math. A window is a UTC calendar month by default. Shared so
 * the API counts and the UI displays the SAME window boundaries.
 */
export function currentMonthlyWindow(now: Date = new Date()): { start: Date; end: Date } {
  const start = new Date(Date.UTC(now.getUTCFullYear(), now.getUTCMonth(), 1));
  const end = new Date(Date.UTC(now.getUTCFullYear(), now.getUTCMonth() + 1, 1));
  return { start, end };
}

/** A stable key for a metric + window, used as the UsageCounter primary key. */
export function usageKey(metric: UsageMetric, windowStart: Date): string {
  return `${metric}:${windowStart.toISOString().slice(0, 10)}`;
}

/** Build a counter DTO + compute exceeded. */
export function toCounterDto(
  metric: UsageMetric,
  used: number,
  limit: number | null,
  window: { start: Date; end: Date },
): UsageCounterDto {
  return {
    metric,
    used,
    limit,
    windowStart: window.start.toISOString(),
    windowEnd: window.end.toISOString(),
    exceeded: limit !== null && used >= limit,
  };
}

/**
 * Would incrementing by `amount` exceed the limit? Fails CLOSED: a null/unknown

```

```

    * limit when `failClosed` is set blocks the increment.
    */
export function wouldExceed(
  used: number,
  limit: number | null,
  amount: number,
  failClosed = false,
): boolean {
  if (limit === null) return failClosed ? true : false;
  return used + amount > limit;
}

```

packages/shared/src/entitlements/schema.ts — SAFE_DIRECT_COPY

```

import { z } from 'zod';

export const usageMetricSchema = z.enum([
  'sessionsStarted',
  'concurrentSessions',
  'recordedMinutes',
  'fileTransferBytes',
]);

export const incrementUsageSchema = z.object({
  metric: usageMetricSchema,
  amount: z.number().int().positive().max(1_000_000_000),
});

export type IncrementUsageInput = z.infer<typeof incrementUsageSchema>;

```

packages/shared/src/entitlements/index.ts — SAFE_DIRECT_COPY

```

export * from './types';
export * from './planLimits';
export * from './featureGate';
export * from './usageWindow';
export * from './schema';

```

packages/shared/src/index.ts — REVIEW_REQUIRED (patch)

```

// ...existing exports...
export * from './entitlements';

```

packages/shared/src/errors/errorCodes.ts — REVIEW_REQUIRED (patch)

```

export const ErrorCodes = {
  // ...existing codes...
  FEATURE_NOT_ENTITLED: 'FEATURE_NOT_ENTITLED',
  USAGE_LIMIT_EXCEEDED: 'USAGE_LIMIT_EXCEEDED',
  STRIPE_SIGNATURE_INVALID: 'STRIPE_SIGNATURE_INVALID',
} as const;

```

Part B — API: Prisma schema + migration

apps/api/prisma/schema.prisma — REVIEW_REQUIRED (patch)

Append these four models. They link to existing `User` and `Subscription` models where useful. Run `prisma generate` after merging.

```
model EntitlementGrant {
  id          String   @id @default(cuid())
  subscriptionId String?
  userId      String?
  tier         String   // PlanTier
  /// Optional per-feature override list (JSON array of FeatureKey).
  featureOverrides Json?
  source      String   // "plan" | "manual" | "trial"
  createdAt   DateTime @default(now())
  expiresAt   DateTime?

  subscription Subscription? @relation(fields: [subscriptionId], references: [id], onDelete: Cascade)
  user        User?          @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([subscriptionId])
  @@index([userId])
}

model UsageCounter {
  id          String   @id @default(cuid())
  subscriptionId String
  metric      String   // UsageMetric
  windowStart DateTime
  windowEnd   DateTime
  used        BigInt   @default(0)
  updatedAt   DateTime @updatedAt

  subscription Subscription @relation(fields: [subscriptionId], references: [id], onDelete: Cascade)

  @@unique([subscriptionId, metric, windowStart])
  @@index([subscriptionId])
}

model BillingAuditEvent {
  id          String   @id @default(cuid())
  subscriptionId String?
  userId      String?
  action      String   // e.g. "entitlements.resolved", "usage.incremented", "stripe.webhook.verify_failed"
  /// Content-light detail. NEVER stores secrets/signatures/keys.
  detail      Json?
  createdAt   DateTime @default(now())

  @@index([subscriptionId])
  @@index([userId])
  @@index([action, createdAt])
}

model StripeWebhookEvent {
  id          String   @id @default(cuid())
  /// Stripe event id (idempotency key). Unique to dedupe retries.
  stripeEventId String   @unique
  type        String
  /// Whether the signature verified. Failures stored WITHOUT the raw signature.
  verified    Boolean
```

```

processedAt    DateTime?
createdAt      DateTime @default(now())

@@index([type])
}

```

Back-relations to add on existing models (REVIEW_REQUIRED):

```

// model Subscription { ... add: }
entitlementGrants EntitlementGrant[]
usageCounters     UsageCounter[]

// model User { ... add: }
entitlementGrants EntitlementGrant[]

```

apps/api/prisma/migrations/0013_billing_entitlements/migration.sql — SAFE_DIRECT_COPY

```

-- EntitlementGrant
CREATE TABLE "EntitlementGrant" (
  "id" TEXT PRIMARY KEY,
  "subscriptionId" TEXT REFERENCES "Subscription"("id") ON DELETE CASCADE,
  "userId" TEXT REFERENCES "User"("id") ON DELETE CASCADE,
  "tier" TEXT NOT NULL,
  "featureOverrides" JSONB,
  "source" TEXT NOT NULL,
  "createdAt" TIMESTAMP NOT NULL DEFAULT now(),
  "expiresAt" TIMESTAMP
);
CREATE INDEX "EntitlementGrant_subscriptionId_idx" ON "EntitlementGrant"("subscriptionId");
CREATE INDEX "EntitlementGrant_userId_idx" ON "EntitlementGrant"("userId");

-- UsageCounter
CREATE TABLE "UsageCounter" (
  "id" TEXT PRIMARY KEY,
  "subscriptionId" TEXT NOT NULL REFERENCES "Subscription"("id") ON DELETE CASCADE,
  "metric" TEXT NOT NULL,
  "windowStart" TIMESTAMP NOT NULL,
  "windowEnd" TIMESTAMP NOT NULL,
  "used" BIGINT NOT NULL DEFAULT 0,
  "updatedAt" TIMESTAMP NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX "UsageCounter_sub_metric_window_key" ON "UsageCounter"("subscriptionId", "metric", "windowStart");
CREATE INDEX "UsageCounter_subscriptionId_idx" ON "UsageCounter"("subscriptionId");

-- BillingAuditEvent
CREATE TABLE "BillingAuditEvent" (
  "id" TEXT PRIMARY KEY,
  "subscriptionId" TEXT,
  "userId" TEXT,
  "action" TEXT NOT NULL,
  "detail" JSONB,
  "createdAt" TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "BillingAuditEvent_subscriptionId_idx" ON "BillingAuditEvent"("subscriptionId");
CREATE INDEX "BillingAuditEvent_userId_idx" ON "BillingAuditEvent"("userId");
CREATE INDEX "BillingAuditEvent_action_createdAt_idx" ON "BillingAuditEvent"("action", "createdAt");

```



```
-- StripeWebhookEvent
CREATE TABLE "StripeWebhookEvent" (
  "id"          TEXT PRIMARY KEY,
  "stripeEventId" TEXT NOT NULL UNIQUE,
  "type"        TEXT NOT NULL,
  "verified"     BOOLEAN NOT NULL,
  "processedAt"  TIMESTAMP,
  "createdAt"    TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "StripeWebhookEvent_type_idx" ON "StripeWebhookEvent"("type");
```

Part C — API: lib, repository, services, routes

apps/api/src/lib/stripeSignature.ts — SAFE_DIRECT_COPY (isolated verify stub)

```
import { createHmac, timingSafeEqual } from 'node:crypto';

/**
 * ISOLATED Stripe signature verification. This is the ONLY place signature logic
 * lives, so it can be swapped for the real `stripe.webhooks.constructEvent` later
 * without touching the route/service.
 *
 * Dev mode: if no signing secret is configured, verification is SKIPPED and the
 * event is treated as unverified (verified=false) but still parsed — so the app
 * runs without real Stripe keys. It NEVER fabricates a verified=true result
 * without a real secret + matching signature.
 */
export interface StripeVerifyResult {
  verified: boolean;
  /** Parsed event id + type (best-effort). Never includes the signature. */
  eventId: string | null;
  type: string | null;
}

export function verifyStripeSignature(
  rawBody: string,
  signatureHeader: string | undefined,
  signingSecret: string | undefined,
): StripeVerifyResult {
  const parsed = safeParse(rawBody);
  const eventId = parsed?.id ?? null;
  const type = parsed?.type ?? null;

  // Dev: no secret configured -> cannot verify. Treat as unverified, not fatal.
  if (!signingSecret) {
    return { verified: false, eventId, type };
  }
  if (!signatureHeader) {
    return { verified: false, eventId, type };
  }

  // Minimal Stripe-style "t=...,v1=..." parsing + HMAC-SHA256 check.
  const { t, v1 } = parseSigHeader(signatureHeader);
  if (!t || !v1) return { verified: false, eventId, type };
```

```

const expected = createHmac('sha256', signingSecret)
  .update(`${t}.${rawBody}`)
  .digest('hex');

const verified = safeEqualHex(expected, v1);
return { verified, eventId, type };
}

function parseSigHeader(header: string): { t: string | null; v1: string | null } {
  let t: string | null = null;
  let v1: string | null = null;
  for (const part of header.split(',')) {
    const [k, val] = part.split('=');
    if (k === 't') t = val;
    if (k === 'v1') v1 = val;
  }
  return { t, v1 };
}

function safeEqualHex(a: string, b: string): boolean {
  try {
    const ba = Buffer.from(a, 'hex');
    const bb = Buffer.from(b, 'hex');
    return ba.length === bb.length && timingSafeEqual(ba, bb);
  } catch {
    return false;
  }
}

function safeParse(raw: string): { id?: string; type?: string } | null {
  try {
    return JSON.parse(raw);
  } catch {
    return null;
  }
}

```

apps/api/src/lib/entitlements.ts — SAFE_DIRECT_COPY

```

import {
  resolveEntitlements,
  type EntitlementSet,
  type PlanTier,
} from '@remotedesk/shared';

/**
 * Resolve a subscription's entitlements. If the tier is missing/unknown we fall
 * back to the fail-closed plan and flag it, so callers can enforce conservatively.
 */
export function resolveForTier(tier: string | null | undefined): EntitlementSet {
  const known: PlanTier[] = ['free', 'pro', 'business', 'enterprise'];
  if (tier && (known as string[]).includes(tier)) {
    return resolveEntitlements(tier as PlanTier, false);
  }
  return resolveEntitlements('free', true); // fail closed
}

```

```

import {
  currentMonthlyWindow,
  toCounterDto,
  wouldExceed,
  type UsageCounterDto,
  type UsageMetric,
} from '@remotedesk/shared';
import { prisma } from '../db/prisma';

/**
 * Usage counter helpers. Increments are atomic via Prisma upsert + BigInt add.
 * Limits are passed in by the caller (resolved from entitlements) so this stays
 * a pure persistence concern.
 */
export const usageCounters = {
  async read(
    subscriptionId: string,
    metric: UsageMetric,
    limit: number | null,
  ): Promise<UsageCounterDto> {
    const window = currentMonthlyWindow();
    const row = await prisma.usageCounter.findUnique({
      where: {
        subscriptionId_metric_windowStart: {
          subscriptionId,
          metric,
          windowStart: window.start,
        },
      },
    });
    const used = row ? Number(row.used) : 0;
    return toCounterDto(metric, used, limit, window);
  },

  /**
   * Increment with a fail-closed check. Returns the updated counter. Throws via
   * the caller if it would exceed (caller decides). Here we just enforce + write.
   */
  async increment(
    subscriptionId: string,
    metric: UsageMetric,
    amount: number,
    limit: number | null,
    failClosed: boolean,
  ): Promise<{ counter: UsageCounterDto; exceeded: boolean }> {
    const window = currentMonthlyWindow();
    const current = await this.read(subscriptionId, metric, limit);
    if (wouldExceed(current.used, limit, amount, failClosed)) {
      return { counter: current, exceeded: true };
    }
    const row = await prisma.usageCounter.upsert({
      where: {
        subscriptionId_metric_windowStart: {
          subscriptionId,
          metric,

```

```

        windowStart: window.start,
      },
    },
    create: {
      subscriptionId,
      metric,
      windowStart: window.start,
      windowEnd: window.end,
      used: BigInt(amount),
    },
    update: { used: { increment: BigInt(amount) } },
  });
  return {
    counter: toCounterDto(metric, Number(row.used), limit, window),
    exceeded: false,
  };
},
};
};

```

apps/api/src/lib/billingAudit.ts — SAFE_DIRECT_COPY

```

import { prisma } from '../db/prisma';

/**
 * Billing audit writer. Detail is content-light and MUST NOT contain secrets,
 * signatures, or keys. A small allow-list scrub strips anything that looks
 * sensitive before persistence (defense in depth).
 */
const FORBIDDEN = [/secret/i, /signature/i, /authorization/i, /api[-_]?key/i, /token/i, /password/i];

function scrub(detail: Record<string, unknown> | undefined): Record<string, unknown> | undefined {
  if (!detail) return undefined;
  const out: Record<string, unknown> = {};
  for (const [k, v] of Object.entries(detail)) {
    out[k] = FORBIDDEN.some((re) => re.test(k)) ? '[REDACTED]' : v;
  }
  return out;
}

export const billingAudit = {
  async record(input: {
    subscriptionId?: string | null;
    userId?: string | null;
    action: string;
    detail?: Record<string, unknown>;
  }): Promise<void> {
    await prisma.billingAuditEvent.create({
      data: {
        subscriptionId: input.subscriptionId ?? null,
        userId: input.userId ?? null,
        action: input.action,
        detail: (scrub(input.detail) as unknown as object) ?? undefined,
      },
    });
  },
};

```

```

import { prisma } from '../db/prisma';

export const billingRepository = {
  /** Resolve the active tier string for a subscription, or null. */
  async tierForSubscription(subscriptionId: string): Promise<string | null> {
    const grant = await prisma.entitlementGrant.findFirst({
      where: { subscriptionId },
      orderBy: { createdAt: 'desc' },
    });
    if (grant) return grant.tier;
    const sub = await prisma.subscription.findUnique({ where: { id: subscriptionId } });
    return (sub as { tier?: string } | null)?.tier ?? null;
  },

  /** Idempotent record of a webhook event. Returns false if already seen. */
  async recordWebhookEvent(input: {
    stripeEventId: string;
    type: string;
    verified: boolean;
  }): Promise<boolean> {
    try {
      await prisma.stripeWebhookEvent.create({ data: input });
      return true;
    } catch {
      return false; // unique violation -> duplicate delivery
    }
  },

  async markProcessed(stripeEventId: string): Promise<void> {
    await prisma.stripeWebhookEvent.update({
      where: { stripeEventId },
      data: { processedAt: new Date() },
    }).catch(() => undefined);
  },
};

```

```

import {
  AppError,
  ErrorCodes,
  type EntitlementSet,
  type UsageCounterDto,
  type UsageMetric,
} from '@remotedesk/shared';
import { resolveForTier } from '../../../lib/entitlements';
import { usageCounters } from '../../../lib/usageCounters';
import { billingAudit } from '../../../lib/billingAudit';
import { billingRepository } from '../../../repositories/billingRepository';

export const entitlementsService = {
  async forSubscription(subscriptionId: string): Promise<EntitlementSet> {
    const tier = await billingRepository.tierForSubscription(subscriptionId);
    const entitlements = resolveForTier(tier);
    await billingAudit.record({

```

```

        subscriptionId,
        action: 'entitlements.resolved',
        detail: { tier: entitlements.tier, failClosed: entitlements.failClosed },
    });
    return entitlements;
},

async usage(subscriptionId: string): Promise<UsageCounterDto[]> {
    const entitlements = await this.forSubscription(subscriptionId);
    const metrics: UsageMetric[] = [
        'sessionsStarted',
        'concurrentSessions',
        'recordedMinutes',
        'fileTransferBytes',
    ];
    return Promise.all(
        metrics.map((m) => usageCounters.read(subscriptionId, m, entitlements.limits[m])),
    );
},

async increment(
    subscriptionId: string,
    metric: UsageMetric,
    amount: number,
): Promise<UsageCounterDto> {
    const entitlements = await this.forSubscription(subscriptionId);
    const { counter, exceeded } = await usageCounters.increment(
        subscriptionId,
        metric,
        amount,
        entitlements.limits[metric],
        entitlements.failClosed,
    );
    await billingAudit.record({
        subscriptionId,
        action: 'usage.incremented',
        detail: { metric, amount, exceeded },
    });
    if (exceeded) throw new AppError(ErrorCodes.USAGE_LIMIT_EXCEEDED);
    return counter;
},
};

```

apps/api/src/services/billing/stripeWebhook.service.ts — SAFE_DIRECT_COPY

```

import { verifyStripeSignature } from '../..lib/stripeSignature';
import { billingAudit } from '../..lib/billingAudit';
import { billingRepository } from '../..repositories/billingRepository';

/**
 * Handle a raw Stripe webhook. Verification is delegated to the isolated stub.
 * Failed verification is AUDIT LOGGED (without the signature/secret) and the
 * request is rejected. No real keys are required to run in dev — unverified
 * events are recorded as verified=false and not processed.
 */
export const stripeWebhookService = {
    async handle(input: {

```

```

    rawBody: string;
    signatureHeader: string | undefined;
    signingSecret: string | undefined;
  }): Promise<{ accepted: boolean; verified: boolean }> {
    const result = verifyStripeSignature(
      input.rawBody,
      input.signatureHeader,
      input.signingSecret,
    );

    if (!result.eventId) {
      await billingAudit.record({
        action: 'stripe.webhook.unparseable',
        detail: { verified: result.verified },
      });
      return { accepted: false, verified: result.verified };
    }

    if (!result.verified) {
      // Never store the raw signature or secret — only the outcome.
      await billingAudit.record({
        action: 'stripe.webhook.verify_failed',
        detail: { eventId: result.eventId, type: result.type },
      });
      await billingRepository.recordWebhookEvent({
        stripeEventId: result.eventId,
        type: result.type ?? 'unknown',
        verified: false,
      });
      return { accepted: false, verified: false };
    }

    const isNew = await billingRepository.recordWebhookEvent({
      stripeEventId: result.eventId,
      type: result.type ?? 'unknown',
      verified: true,
    });
    if (isNew) {
      // Real plan-sync handlers wire here in a later pack (no fake completion now).
      await billingRepository.markProcessed(result.eventId);
      await billingAudit.record({
        action: 'stripe.webhook.received',
        detail: { eventId: result.eventId, type: result.type },
      });
    }
    return { accepted: true, verified: true };
  },
};

```

apps/api/src/routes/billing.routes.ts — SAFE_DIRECT_COPY

```

import { Router, raw } from 'express';
import { incrementUsageSchema } from '@remotedesk/shared';
import { requireAuth } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { entitlementsService } from '../services/billing/entitlements.service';
import { stripeWebhookService } from '../services/billing/stripeWebhook.service';

```

```

const h =
  (fn: (req: any, res: any) => Promise<void>) =>
    (req: any, res: any, next: any) =>
      fn(req, res).catch(next);

/** Resolve the subscriptionId from org context. Adjust to your model. */
function subId(req: any): string {
  return (req as OrgRequest & { subscriptionId?: string }).subscriptionId
    ?? (req as OrgRequest).orgId; // fallback: org maps 1:1 to subscription
}

/**
 * /api/billing
 * GET /entitlements -> resolved entitlement set
 * GET /usage -> usage counters for the current window
 * POST /usage/increment -> increment a metric (fail-closed enforcement)
 * POST /stripe/webhook -> raw-body Stripe webhook (isolated verify stub)
 */
export const billingRoutes = Router()
  .get('/entitlements', requireAuth, orgContext, h(async (req, res) => {
    res.json(await entitlementsService.forSubscription(subId(req)));
  })))
  .get('/usage', requireAuth, orgContext, h(async (req, res) => {
    res.json(await entitlementsService.usage(subId(req)));
  })))
  .post('/usage/increment', requireAuth, orgContext, h(async (req, res) => {
    const { metric, amount } = incrementUsageSchema.parse(req.body);
    res.json(await entitlementsService.increment(subId(req), metric, amount));
  })))
  // Stripe needs the raw body for signature verification — mount raw parser here only.
  .post('/stripe/webhook', raw({ type: 'application/json' }), h(async (req, res) => {
    const rawBody = Buffer.isBuffer(req.body) ? req.body.toString('utf8') : '';
    const result = await stripeWebhookService.handle({
      rawBody,
      signatureHeader: req.header('stripe-signature'),
      signingSecret: process.env.STRIPE_WEBHOOK_SECRET, // unset in dev -> unverified
    });
    res.status(result.accepted ? 200 : 400).json(result);
  }));

```

apps/api/src/server.ts — REVIEW_REQUIRED (patch)

```

// ...existing imports...
import { billingRoutes } from '../routes/billing.routes';

// ...inside createServer(), next to the other app.use('/api/...') mounts:
app.use('/api/billing', billingRoutes);

```

Part D — Web: entitlements + usage pages

apps/web/app/dashboard/billing/billingApi.ts — SAFE_DIRECT_COPY

```

import type { EntitlementSet, UsageCounterDto, UsageMetric } from '@remotedesk/shared';
import { api } from '../../src/api/apiClient';

```



```
export const billingApi = {
  entitlements: () => api.get<EntitlementSet>('/billing/entitlements'),
  usage: () => api.get<UsageCounterDto[]>('/billing/usage'),
  increment: (metric: UsageMetric, amount: number) =>
    api.post<UsageCounterDto>('/billing/usage/increment', { metric, amount }),
};
```

apps/web/app/dashboard/billing/entitlements/page.tsx — SAFE_DIRECT_COPY

```
'use client';

import React, { useEffect, useState } from 'react';
import {
  PLAN_DEFINITIONS,
  blockedFeatures,
  type EntitlementSet,
  type FeatureKey,
} from '@remotedesk/shared';
import { billingApi } from '../billingApi';
import styles from '../billing.module.css';

const ALL_FEATURES: FeatureKey[] = [
  'fileTransfer', 'clipboardSync', 'unattendedAccess',
  'sessionRecording', 'multiMonitor', 'auditExport',
];

export default function EntitlementsPage() {
  const [ent, setEnt] = useState<EntitlementSet | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    billingApi.entitlements().then(setEnt).catch(() => setError('Could not load entitlements.'));
  }, []);

  if (error) return <main className={styles.page}><div className={styles.error}>{error}</div></main>;
  if (!ent) return <main className={styles.page}><p className={styles.muted}>Loading...</p></main>;

  const plan = PLAN_DEFINITIONS[ent.tier];
  const blocked = blockedFeatures(ent, ALL_FEATURES);

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Plan & entitlements</h1>
        <p className={styles.sub}>
          Current plan: <strong>{plan.name}</strong>
          {ent.failClosed && <span className={styles.warn}> · fail-closed fallback active</span>}
        </p>
      </header>

      <section className={styles.panel}>
        <h2>Features</h2><ul className={styles.featureList}>
          {ALL_FEATURES.map((f) => (
            <li key={f} data-on={ent.features.includes(f)}>
              {ent.features.includes(f) ? '✓' : '☒'} {f}
            </li>
          ))}
        </ul>
      </section>
    </main>
  );
}
```

```

    )))
  </ul>
  {blocked.length > 0 && (
    <p className={styles.muted}>Blocked on this plan: {blocked.join(', ')}</p>
  )}
</section>

<section className={styles.panel}>
  <h2>Limits</h2>
  <table className={styles.table}>
    <thead><tr><th>Metric</th><th>Limit</th></tr></thead>
    <tbody>
      {Object.entries(ent.limits).map(([metric, limit]) => (
        <tr key={metric}>
          <td>{metric}</td>
          <td>{limit === null ? 'Unlimited' : limit.toLocaleString()}</td>
        </tr>
      ))}
    </tbody>
  </table>
</section>
</main>
);
}

```

apps/web/app/dashboard/billing/usage/page.tsx — SAFE_DIRECT_COPY

```

'use client';

import React, { useEffect, useState } from 'react';
import type { UsageCounterDto } from '@remotedesk/shared';
import { billingApi } from '../../billingApi';
import styles from '../../billing.module.css';

export default function UsagePage() {
  const [counters, setCounters] = useState<UsageCounterDto[]>([]);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    billingApi.usage().then(setCounters).catch(() => setError('Could not load usage.'));
  }, []);

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Usage this month</h1>
        <p className={styles.sub}>Counts reset at the start of each UTC month.</p>
      </header>

      {error && <div className={styles.error}>{error}</div>}

      <section className={styles.panel}>
        <table className={styles.table}>
          <thead>
            <tr><th>Metric</th><th>Used</th><th>Limit</th><th>Status</th></tr>
          </thead>
          <tbody>

```

```

    {counters.map((c) => (
      <tr key={c.metric}>
        <td>{c.metric}</td>
        <td>{c.used.toLocaleString()}</td>
        <td>{c.limit === null ? 'Unlimited' : c.limit.toLocaleString()}</td>
        <td>
          <span className={styles.badge} data-exceeded={c.exceeded}>
            {c.exceeded ? 'Limit reached' : 'OK'}
          </span>
        </td>
      </tr>
    ))}
  </tbody>
</table>
</section>
</main>
);
}

```

apps/web/app/dashboard/billing/billing.module.css — SAFE_DIRECT_COPY

```

.page { padding: 24px; max-width: 1000px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 22px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 20px; }
.warn { color: #b45309; }
.error { background: #fef2f2; color: #991b1b; padding: 10px 14px; border-radius: 8px; }
.muted { color: var(--muted, #6b7280); font-size: 13px; }
.panel { border: 1px solid var(--border, #e5e7eb); border-radius: 12px; padding: 16px 18px; margin-bottom: 16px; background: #fff; }
.panel h2 { font-size: 15px; margin: 0 0 12px; }
.featureList { list-style: none; padding: 0; margin: 0; display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 6px; font-size: 13px; }
.featureList li[data-on='false'] { color: #9ca3af; }
.table { width: 100%; border-collapse: collapse; font-size: 13px; }
.table th, .table td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--border, #e5e7eb); }
.table th { color: var(--muted, #6b7280); font-weight: 600; }
.badge { font-size: 12px; padding: 2px 10px; border-radius: 999px; background: #ecfdf5; color: #065f46; }
.badge[data-exceeded='true'] { background: #fef2f2; color: #991b1b; }

```

apps/web/app/dashboard/page.tsx — REVIEW_REQUIRED (patch)

```

// ...existing imports...
import Link from 'next/link';

// ...inside the dashboard grid:
<Link href="/dashboard/billing/entitlements" className="dashboard-card">
  <h3>Plan & entitlements</h3>
  <p>See your plan features and limits.</p>
</Link>
<Link href="/dashboard/billing/usage" className="dashboard-card">
  <h3>Usage</h3>
  <p>Track usage against your plan limits.</p>
</Link>

```

Part E — Desktop: entitlement gating on session start

apps/desktop/src/renderer/src/services/entitlementsClient.ts — SAFE_DIRECT_COPY

```
import { resolveEntitlements, type EntitlementSet } from '@remotedesk/shared';

/**
 * Fetches the entitlement set from the API. FAILS CLOSED: on any error it
 * returns the fail-closed fallback so the desktop disables gated features
 * rather than enabling them optimistically.
 */
export class EntitlementsClient {
  constructor(
    private readonly baseUrl: string,
    private readonly authToken: string,
  ) {}

  async fetch(): Promise<EntitlementSet> {
    try {
      const res = await fetch(`${this.baseUrl}/api/billing/entitlements`, {
        headers: { authorization: `Bearer ${this.authToken}` },
      });
      if (!res.ok) throw new Error(`entitlements fetch failed: ${res.status}`);
      return (await res.json()) as EntitlementSet;
    } catch {
      // Fail closed: most restrictive plan, flagged.
      return resolveEntitlements('free', true);
    }
  }
}
```

apps/desktop/src/renderer/src/services/featureGateClient.ts — SAFE_DIRECT_COPY

```
import { checkFeature, type EntitlementSet, type FeatureKey } from '@remotedesk/shared';

/**
 * Thin client-side gate wrapping the shared pure check. Used to enable/disable
 * UI affordances. Defaults to DENY for unknown entitlements.
 */
export class FeatureGateClient {
  constructor(private entitlements: EntitlementSet) {}

  update(entitlements: EntitlementSet): void {
    this.entitlements = entitlements;
  }

  allows(feature: FeatureKey): boolean {
    return checkFeature(this.entitlements, feature).allowed;
  }
}
```

apps/desktop/src/renderer/src/session/sessionStart.ts — REVIEW_REQUIRED (patch)

Gate file transfer / clipboard / unattended access UI on entitlements before enabling them. **Native input execution stays disabled — this only toggles UI affordances.**

```
// ...existing imports...
import { EntitlementsClient } from '../services/entitlementsClient';
import { FeatureGateClient } from '../services/featureGateClient';

// --- during session start, before exposing gated UI ---
const entitlements = await new EntitlementsClient(apiBase, authToken).fetch();
const gate = new FeatureGateClient(entitlements);

// Enable/disable UI affordances (fail closed if entitlements unavailable):
setFileTransferEnabled(gate.allows('fileTransfer'));
setClipboardSyncEnabled(gate.allows('clipboardSync'));
setUnattendedAccessEnabled(gate.allows('unattendedAccess'));

if (entitlements.failClosed) {
  showBanner('Running with limited features — could not verify your plan.');
```

```
}
```

Part F — Tests

tests/entitlements/featureGate.test.ts — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { resolveEntitlements, checkFeature, blockedFeatures } from '@remotedesk/shared';

describe('featureGate', () => {
  it('allows features included in the plan', () => {
    const pro = resolveEntitlements('pro');
    expect(checkFeature(pro, 'fileTransfer').allowed).toBe(true);
  });

  it('denies features not in the plan with reason not_in_plan', () => {
    const free = resolveEntitlements('free');
    const r = checkFeature(free, 'fileTransfer');
    expect(r.allowed).toBe(false);
    expect(r.reason).toBe('not_in_plan');
  });

  it('fails closed: unknown tier denies with reason fail_closed', () => {
    const fc = resolveEntitlements('free', true);
    const r = checkFeature(fc, 'unattendedAccess');
    expect(r.allowed).toBe(false);
    expect(r.reason).toBe('fail_closed');
  });

  it('lists blocked features', () => {
    const free = resolveEntitlements('free');
    expect(blockedFeatures(free, ['fileTransfer', 'clipboardSync'])).toEqual(['fileTransfer']);
  });
});
```

tests/entitlements/planLimits.test.ts — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { PLAN_DEFINITIONS, getPlan } from '@remotedesk/shared';
```

```

describe('planLimits', () => {
  it('free is the most restrictive paid-feature-free tier', () => {
    expect(PLAN_DEFINITIONS.free.features).not.toContain('fileTransfer');
    expect(PLAN_DEFINITIONS.free.limits.fileTransferBytes).toBe(0);
  });

  it('enterprise has unlimited limits', () => {
    expect(PLAN_DEFINITIONS.enterprise.limits.sessionsStarted).toBeNull();
    expect(PLAN_DEFINITIONS.enterprise.limits.concurrentSessions).toBeNull();
  });

  it('getPlan falls back to free for an unknown tier', () => {
    // @ts-expect-error testing fallback
    expect(getPlan('nope').tier).toBe('free');
  });

  it('business unlocks audit export and recording', () => {
    expect(PLAN_DEFINITIONS.business.features).toContain('auditExport');
    expect(PLAN_DEFINITIONS.business.features).toContain('sessionRecording');
  });
});

```

tests/entitlements/usageWindow.test.ts — SAFE_DIRECT_COPY

```

import { describe, it, expect } from 'vitest';
import { currentMonthlyWindow, usageKey, toCounterDto, wouldExceed } from '@remotedesk/shared';

describe('usageWindow', () => {
  it('builds a UTC month window', () => {
    const { start, end } = currentMonthlyWindow(new Date('2026-06-15T10:00:00Z'));
    expect(start.toISOString()).toBe('2026-06-01T00:00:00.000Z');
    expect(end.toISOString()).toBe('2026-07-01T00:00:00.000Z');
  });

  it('derives a stable usage key', () => {
    const { start } = currentMonthlyWindow(new Date('2026-06-15T10:00:00Z'));
    expect(usageKey('sessionsStarted', start)).toBe('sessionsStarted:2026-06-01');
  });

  it('marks exceeded when used >= limit', () => {
    const w = currentMonthlyWindow();
    expect(toCounterDto('sessionsStarted', 50, 50, w).exceeded).toBe(true);
    expect(toCounterDto('sessionsStarted', 49, 50, w).exceeded).toBe(false);
  });

  it('wouldExceed fails closed on null limit when flagged', () => {
    expect(wouldExceed(0, null, 1, true)).toBe(true);
    expect(wouldExceed(0, null, 1, false)).toBe(false);
    expect(wouldExceed(9, 10, 2, false)).toBe(true);
  });
});

```

tests/billing/webhook-signature-stub.test.ts — SAFE_DIRECT_COPY

```

import { describe, it, expect } from 'vitest';
import { createHmac } from 'node:crypto';

```

```

import { verifyStripeSignature } from '../..apps/api/src/lib/stripeSignature';

const body = JSON.stringify({ id: 'evt_123', type: 'customer.subscription.updated' });

describe('verifyStripeSignature (isolated stub)', () => {
  it('returns unverified (not fatal) when no secret is configured (dev mode)', () => {
    const r = verifyStripeSignature(body, undefined, undefined);
    expect(r.verified).toBe(false);
    expect(r.eventId).toBe('evt_123');
    expect(r.type).toBe('customer.subscription.updated');
  });

  it('verifies a correct HMAC signature', () => {
    const secret = 'whsec_test';
    const t = '1700000000';
    const sig = createHmac('sha256', secret).update(`${t}.${body}`).digest('hex');
    const r = verifyStripeSignature(body, `t=${t},v1=${sig}`, secret);
    expect(r.verified).toBe(true);
  });

  it('rejects a tampered signature', () => {
    const r = verifyStripeSignature(body, 't=1700000000,v1=deadbeef', 'whsec_test');
    expect(r.verified).toBe(false);
  });

  it('never throws on unparseable body', () => {
    const r = verifyStripeSignature('not-json', undefined, undefined);
    expect(r.eventId).toBeNull();
  });
});

```

tests/billing/usage-counter-contract.test.ts — SAFE_DIRECT_COPY

```

import { describe, it, expect } from 'vitest';
import { incrementUsageSchema, toCounterDto, currentMonthlyWindow } from '@remotedesk/shared';

describe('usage counter contract', () => {
  it('accepts a valid increment payload', () => {
    const parsed = incrementUsageSchema.parse({ metric: 'sessionsStarted', amount: 1 });
    expect(parsed.amount).toBe(1);
  });

  it('rejects zero/negative amounts', () => {
    expect(() => incrementUsageSchema.parse({ metric: 'sessionsStarted', amount: 0 })).toThrow();
    expect(() => incrementUsageSchema.parse({ metric: 'sessionsStarted', amount: -5 })).toThrow();
  });

  it('rejects unknown metrics', () => {
    expect(() => incrementUsageSchema.parse({ metric: 'bogus', amount: 1 })).toThrow();
  });

  it('counter dto reports unlimited as null limit, never exceeded', () => {
    const w = currentMonthlyWindow();
    const dto = toCounterDto('sessionsStarted', 999999, null, w);
    expect(dto.limit).toBeNull();
    expect(dto.exceeded).toBe(false);
  });
});

```

```
});  
});
```

Part G — Docs

docs/billing-entitlements-hardening.md — SAFE_DIRECT_COPY

Billing Hardening: Entitlements, Usage Limits & Audit-Safe Plan Enforcement

A Stripe-**ready** entitlements + usage foundation. **This is NOT production-ready billing:** there is no live checkout, no portal, and the Stripe webhook verifier is an isolated stub. No real Stripe secret keys are required to run in dev, and no payment is ever faked as complete.

Plans & features

`PLAN\_DEFINITIONS` in `@remotedesk/shared` is the single source of truth for tier features + numeric limits (free / pro / business / enterprise). Stripe product/price IDs are intentionally absent. `resolveEntitlements(tier)` and `checkFeature()` run identically on the API (enforcement) and desktop/web (UI gating).

Fail-closed enforcement

- Unknown/unresolvable tier → `resolveForTier` returns the **free** plan flagged `failClosed: true`.
- `checkFeature` on a fail-closed set denies anything outside the safe fallback with reason `fail\_closed`.
- `wouldExceed(used, null, amount, failClosed=true)` blocks increments when the limit is unknown.
- Desktop `EntitlementsClient.fetch()` returns the fail-closed set on ANY error, so gated UI (file transfer / clipboard / unattended) is disabled, not enabled.

Endpoints

- `GET /api/billing/entitlements` — resolved entitlement set (audited).
- `GET /api/billing/usage` — usage counters for the current UTC-month window.
- `POST /api/billing/usage/increment` — increment a metric; throws `USAGE\_LIMIT\_EXCEEDED` when over.
- `POST /api/billing/stripe/webhook` — raw-body webhook; verification via the isolated stub.

Stripe webhook (isolated stub)

`lib/stripeSignature.ts` is the **ONLY** place signature logic lives. In dev (no `STRIPE\_WEBHOOK\_SECRET`) events are parsed but recorded `verified: false` and not processed. With a secret, it does a real HMAC-SHA256 check. **Failed verification is audit-logged (event id + type only — never the signature or secret)** and the request is rejected. Events are de-duped by `stripeEventId`.

Audit safety

`billingAudit.record()` scrubs any detail key matching secret/signature/authorization/api key/token/password before persistence. No secrets are ever written.

Not done yet (see manifest knownLimitations)

Live checkout, billing portal, and real plan-sync handlers are NOT wired. Native input execution remains disabled.

Part H — Handoff meta (doNotMerge)

generated-remotedesk-pack13-merge-summary.md — doNotMerge

RemoteDesk Pack 13 — Merge Summary

****Pack:**** Billing Hardening — Stripe-Ready Entitlements, Usage Limits & Audit-Safe Plan Enforcement

****Files:**** 38 (29 SAFE_DIRECT_COPY, 6 REVIEW_REQUIRED, 3 doNotMerge meta)

Build order

1. `packages/shared` — entitlements/* then patch shared `index.ts` + `errors/errorCodes.ts`; build.
2. `apps/api` — patch `schema.prisma` (4 models + back-relations), run `0013\_billing\_entitlements` migration + `prisma generate`, drop in lib/repo/services/routes, patch `server.ts`.
3. `apps/web` — drop in `dashboard/billing/\*`, patch dashboard links.
4. `apps/desktop` — drop in the two clients, patch `session/sessionStart.ts`.
5. Tests — `pnpm -r test`.

REVIEW_REQUIRED (hand-merge)

- `packages/shared/src/index.ts` — add entitlements barrel.
- `packages/shared/src/errors/errorCodes.ts` — add 3 codes.
- `apps/api/prisma/schema.prisma` — add 4 models + back-relations on Subscription/User.
- `apps/api/src/server.ts` — mount /api/billing.
- `apps/web/app/dashboard/page.tsx` — add 2 links.
- `apps/desktop/src/renderer/src/session/sessionStart.ts` — gate UI on entitlements.

Known limitations

- No live Stripe checkout or billing portal wired (entitlements/usage only).
- Webhook verifier is an isolated stub; swap for `stripe.webhooks.constructEvent` + real secret for prod.
- `subId()` assumes org maps 1:1 to subscription; wire the real subscription lookup.
- Real plan-sync on verified webhook is a no-op placeholder (no fake completion).
- Native input execution remains disabled by design.

Safety

- No real Stripe keys; dev runs with verification skipped (verified=false), never faked true.
- Failed verification is audit-logged WITHOUT the signature/secret.
- Plan enforcement fails closed everywhere (server + desktop).

Verify

- `pnpm -r typecheck && pnpm -r test`
- Manual: GET /api/billing/entitlements returns free+failClosed when no grant exists.
- Manual: POST a fake webhook with no secret -> recorded verified=false, not processed, audit row written.

generated-remotedesk-pack13-code-review.md — doNotMerge

RemoteDesk Pack 13 — Code Review Notes

Strengths

- One plan catalog + one gate function shared by API and clients — no drift.
- Signature logic fully isolated in `stripeSignature.ts`; trivial to swap for the real SDK.
- Fail-closed is consistent: unknown tier, unknown limit, and client fetch error all deny.
- Webhook events de-duped by stripeEventId; failures audited without secrets.
- Tests import only real shared/copied files (no REVIEW_REQUIRED-only imports).

Reviewer checklist

- [] Map org/subscription correctly in `subId()` (currently falls back to orgId).
- [] Confirm `Subscription` has a `tier` field or rely on EntitlementGrant rows.
- [] Raw-body parser for the webhook is mounted ONLY on that route; confirm global json() doesn't consume it first.

- [] BigInt usage columns serialize correctly through your API layer (Number() cast used in DTOs).
- [] Decide retention/reset strategy for UsageCounter windows.

Security

- No secret keys in code; `STRIPE\_WEBHOOK\_SECRET` read from env, unset in dev.
- billingAudit scrubs sensitive detail keys before persistence.
- No fake payment completion; verified webhooks currently only de-dupe + audit.

generated-remotedesk-pack13-manifest.json — doNotMerge

```
{
  "pack": "pack13",
  "title": "Billing Hardening: Stripe-Ready Entitlements, Usage Limits & Audit-Safe Plan Enforcement",
  "actualFileCount": 38,
  "safeDirectCopy": [
    "packages/shared/src/entitlements/types.ts",
    "packages/shared/src/entitlements/planLimits.ts",
    "packages/shared/src/entitlements/featureGate.ts",
    "packages/shared/src/entitlements/usageWindow.ts",
    "packages/shared/src/entitlements/schema.ts",
    "packages/shared/src/entitlements/index.ts",
    "apps/api/prisma/migrations/0013_billing_entitlements/migration.sql",
    "apps/api/src/lib/entitlements.ts",
    "apps/api/src/lib/usageCounters.ts",
    "apps/api/src/lib/billingAudit.ts",
    "apps/api/src/lib/stripeSignature.ts",
    "apps/api/src/repositories/billingRepository.ts",
    "apps/api/src/services/billing/entitlements.service.ts",
    "apps/api/src/services/billing/stripeWebhook.service.ts",
    "apps/api/src/routes/billing.routes.ts",
    "apps/web/app/dashboard/billing/billingApi.ts",
    "apps/web/app/dashboard/billing/entitlements/page.tsx",
    "apps/web/app/dashboard/billing/usage/page.tsx",
    "apps/web/app/dashboard/billing/billing.module.css",
    "apps/desktop/src/renderer/src/services/entitlementsClient.ts",
    "apps/desktop/src/renderer/src/services/featureGateClient.ts",
    "tests/entitlements/featureGate.test.ts",
    "tests/entitlements/planLimits.test.ts",
    "tests/entitlements/usageWindow.test.ts",
    "tests/billing/webhook-signature-stub.test.ts",
    "tests/billing/usage-counter-contract.test.ts",
    "docs/billing-entitlements-hardening.md"
  ],
  "reviewRequired": [
    "packages/shared/src/index.ts",
    "packages/shared/src/errors/errorCodes.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/server.ts",
    "apps/web/app/dashboard/page.tsx",
    "apps/desktop/src/renderer/src/session/sessionStart.ts"
  ],
  "doNotMerge": [
    "generated-remotedesk-pack13-merge-summary.md",
    "generated-remotedesk-pack13-code-review.md",
    "generated-remotedesk-pack13-manifest.json"
  ],
  "dependenciesRequired": {
```

```

    "new": [],
    "existing": ["zod", "@prisma/client", "express", "next", "react", "vitest"],
    "assumedFromPriorBatches": [
      "@remotedesk/shared AppError + ErrorCodes",
      "apps/api db/prisma client",
      "apps/api middleware: requireAuth, orgContext",
      "apps/web src/api/apiClient (api.get/post)",
      "existing Prisma Subscription and User models",
      "apps/desktop session start flow exposing setFileTransferEnabled/etc."
    ]
  },
  "knownLimitations": [
    "No live Stripe checkout or billing portal; entitlements + usage only.",
    "Webhook verifier is an isolated stub; swap for stripe.webhooks.constructEvent + real secret for prod.",
    "subId() assumes org maps 1:1 to subscription; wire real lookup.",
    "Verified webhook plan-sync is a no-op placeholder (no fake payment completion).",
    "Native input execution remains disabled."
  ],
  "estimatedCompletionAfterCarefulMerge": "63-69%"
}

```

Pack 13 done: 38 files. A Stripe-**ready** (not Stripe-live) entitlements + usage foundation: one shared plan catalog, a pure feature gate used identically on server and clients, monthly usage counters with atomic increments, and an **isolated** webhook signature verifier that runs keyless in dev and audit-logs failures without storing secrets.

The clean bit: everything fails closed — unknown tier, unknown limit, and a desktop fetch error all deny gated features rather than enabling them. Signature logic lives in exactly one swappable file, so wiring real Stripe later doesn't touch the route or service. No payment is ever faked complete.

Next number = Pack 14 (audit logging + compliance exports + support tickets), already queued below. Bolo.